Adevar Labs

Prepared for:
M0

July 30, 2026

PYUSDx integration Audit Report

Version: after fix review

Table of Contents

1. Executive Summary

About PYUSDx integration

M0 is a stablecoin issuance platform whose base token, M, can be wrapped by permissionless Extensions that customize yield, compliance, and distribution for each issuer. This audit covers two codebases for the PYUSDx integration: the migration of the JMI ("Just Mint It") minting extension from M-backed to PYUSDx-backed collateral as a renamed MultiMint extension that also supports other assets like USDC, and an OFT wrapper making PYUSDx compatible with the LayerZero Stargate bridging UI.

Audit Summary

The table below summarizes identified vulnerabilities by severity and remediation status across both audited repositories.

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	3	1	2

Enhancement opportunities: 4

Overall Security Posture

The review identified no Critical, High, or Medium severity issues. All 3 findings are Low severity and concentrate in operator configuration flows rather than core accounting or token logic. Both codebases are structurally sound, and the residual risk stems from configuration changes that leave mappings, authorizations, or caps partially updated rather than from exploitable logic flaws.

Saturn Dollar

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	1	0	1

Enhancement opportunities: 2

PYUSDx

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	2	1	1

Enhancement opportunities: 2

After Fix Review Summary

The after fix review confirms 1 of the 3 Low severity findings fixed and 2 acknowledged, with 2 of the 4 enhancement opportunities implemented and 2 acknowledged.

The team acknowledged the remaining 2 findings and 2 enhancement opportunities with documented rationale.

The residual risk is therefore concentrated in operator procedure rather than code: the acknowledged items all depend on operator or user error or an off-nominal configuration sequence, and 2 of them are mitigated by actions the team must perform outside the contracts, namely setting M's cap to 0 at the end of the migration and restoring the LayerZero delegate after any operator revocation.

Before Fix Review Summary

The before fix review produced 3 Low severity findings and 4 enhancement opportunities across the 2 repositories, with no Critical, High, or Medium severity issues.

In **saturn-dollar**, 1 Low severity finding concerns the M reserve cap not being decremented alongside the balance in **_replaceAsset**, which reopens M wrapping headroom after each asset replacement and lets M exposure be re-established despite the migration's intent to remove it permanently.

In **PYUSDx**, 2 Low severity findings concern the bridge and portal configuration surface: unvalidated **destinationToken** and **recipient** address shapes ahead of an irreversible burn and a permanently reverting destination-side delivery, and an EID reassignment that desyncs **remotePeer** and misroutes subsequent sends. Both carry medium impact with low likelihood, since they require operator or user error or an unusual reconfiguration sequence to trigger.

The 4 enhancement opportunities address defense in depth and observability. In **saturn-dollar**, **migrate()** leaves **replaceAssetWhitelist** empty, so **replaceAsset** stays open to every caller until a separate configuration transaction lands, and it registers the M reserve without emitting an asset-registration event, leaving log-based consumers inconsistent with on-chain state. In **PYUSDx**, an **OPERATOR_ROLE** revocation clears the LayerZero delegate even when the revoked account is not the delegate, and **_setFallbackRecipient** does not check whether the recipient is frozen on PYUSDx, so a frozen fallback address can brick the frozen-recipient redirect path in **receiveMessage**.

Launch Recommendations

After Fix Review

Every recommendation below was reviewed before launch: 3 were fixed in code and 4 were acknowledged by the team with documented rationale, as recorded per finding in the Findings section.

Before Fix Review

Strongly recommended

- In **PYUSDx**, make bridge chain ID, EID, and remote-peer updates atomic, or clear stale peer state for both reassigned and orphaned chains before enabling sends.
- In **PYUSDx**, validate **destinationToken** and **recipient** as well-formed addresses before burning or dispatching tokens.
- In **PYUSDx**, clear the LayerZero delegate only when the revoked **OPERATOR_ROLE** account is the current delegate.
- In **PYUSDx**, reject a frozen address in **_setFallbackRecipient** so the frozen-recipient redirect in **receiveMessage** cannot brick delivery.
- In **saturn-dollar**, reject M wrapping instead of relying on **cap == balance** to make the migration's removal of M exposure permanent.
- In **saturn-dollar**, configure the authorized reserve replacement caller atomically with the migration, so **replaceAsset** is never open to every caller.
- In **saturn-dollar**, emit an explicit event when migration registers the M reserve so log-based consumers remain synchronized with on-chain state.

Audit Scope

Saturn Dollar

- **Repository:** https://github.com/m0-platform/saturn-dollar
- **Before Fix Review Commit Hash:** 31e1d84a1236b74ffa9141ed49a1d8e464bf4591
- **After Fix Review Commit Hash:** 26507265ea099ebe807a27144e73352510e9e076
- **Files/Modules in Scope:**
  - src/**/*.sol

PYUSDx

- **Repository:** https://github.com/m0-platform/PYUSDx
- **Before Fix Review Commit Hash:** 143aa82b3a25eb57849ab1df85a11834d445bf30
- **After Fix Review Commit Hash:** 0a4f2ffa0a4155b84bd61296453ad51dbecd737f
- **Files/Modules in Scope:**
  - Changes in PR#83

2. Assumptions and Considerations

PYUSDx OFT Wrapper

Project & Protocol Assumptions

1. The wrapper, Portal, and bridge adapter proxies are deployed atomically so no attacker can front-run their unguarded initializers.
2. Destination tokens, peers, chain-ID mappings, bridge adapters, and payload gas limits are configured consistently and symmetrically across all connected chains before routes carry value.
3. The **fallbackRecipient** is kept set to a controlled, non-frozen address capable of custodying redirected funds.
4. The team monitors the bridge and pauses send and/or receive in a timely manner when an issue is detected.

Centralization

- The wrapper **OPERATOR_ROLE** can point any destination EID at an arbitrary destination token or remove a route.
- The Portal **OPERATOR_ROLE** can add, remove, or change bridge adapters and destination gas limits.
- The Portal **PAUSER_ROLE** can halt all outbound and inbound bridging immediately.
- The Portal **DEFAULT_ADMIN_ROLE** sets the **fallbackRecipient** that custodies inbound funds redirected away from frozen recipients.
- The PYUSDx **FREEZE_MANAGER_ROLE**, independent of the wrapper and Portal roles, can freeze any account
- The wrapper, Portal, and adapter are upgradeable proxies whose upgrade authority is a single point of control.

Trust Assumptions

- The protocol trusts the LayerZero Endpoint V2 to deliver messages only from verified configured peers, revert on underpayment, and refund excess native fees.
- The protocol trusts the LayerZero DVN set to honestly verify cross-chain messages, since a compromised DVN set could forge inbound mints.
- The protocol trusts the LayerZero executor to deliver destination messages within a reasonable timeframe using at least the configured gas limit.

Saturn Dollar

Project & Protocol Assumptions

1. USDat Unwrap operations are capped by PYUSDx backing, which starts at 0 after the migration. Until an authorized party runs **replaceAsset** to migrate the M reserve and/or organic PYUSDx inflows arrive, holders cannot unwrap.
2. The configured yield recipient is a valid, non-frozen address able to hold USDat, otherwise yield claiming reverts and funds are stranded.
3. **migrate()** is permissionless (one-shot reinitializer) and **replaceAsset** is open by default, so both must be controlled through correct operational sequencing rather than in-contract access control.

Centralization

- The **DEFAULT_ADMIN_ROLE** can grant and revoke every other role.
- The **FREEZE_MANAGER_ROLE** can freeze any account, immobilizing its funds from being wrapped, unwrapped, transferred, or received.
- The **FORCED_TRANSFER_MANAGER_ROLE** can seize tokens from any frozen account and send them to an arbitrary recipient.
- The **WHITELIST_MANAGER_ROLE** can enable the whitelist and remove any account, blocking that account from all token movement.
- The **ASSET_CAP_MANAGER_ROLE** controls which alt-assets back the token, their caps, and who may call **replaceAsset**.
- The **YIELD_RECIPIENT_MANAGER_ROLE** both claims yield and redirects the yield destination via **setYieldRecipient**, coupling those powers on a single key.
- The **PAUSER_ROLE** can pause all wrapping, unwrapping, and transfers, halting redemption and stranding users.
- The ProxyAdmin owner can replace the entire USDat implementation and arbitrarily change its logic, mitigated by a 5-day **SaturnTimelock** delay before any upgrade executes.

Trust Assumptions

- The protocol relies on off-chain keepers to claim yield in a timely manner and on a reserve-conversion operator to run **replaceAsset** so that redemptions can resume.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions convey our team's understanding of both the potential consequences of a vulnerability and how likely it is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements–typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue–particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easily an attacker can discover and exploit a vulnerability, as well as how economically attractive the exploit is.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risk.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but the financial risk remains manageable.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically infeasible or requires highly specialized conditions. They may require extraordinary effort or entail significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact with high likelihood (e.g., a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Medium impact with medium likelihood, or high impact with low likelihood

- **Low**: Low impact (regardless of likelihood), or medium impact with low likelihood

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.

4. Findings

L01: M wrapping re-enables after any replaceAsset because the cap is not reduced with the balance

LOCATION

saturn-dollar/31e1d84a/src/USDat.sol:L95

RELEVANT SNIPPET

>_ USDat.sol — SOLIDITY

```
93:     MultiMintStorage storage $ = _getMultiMintStorage();
94:
95:     $.assets[M_TOKEN] = Asset({cap: mBalance, balance: UIntMath.safe240(mBalance), dec
        imals: 6});
96:     $.totalAssets += mBalance;
97: }
```

DESCRIPTION

migrate() registers M with cap == balance, so isAllowedToWrap(M, amount) is false and M cannot be wrapped, the intended permanent state.

_replaceAsset decrements balance but never cap, so each drain reopens cap - balance of wrap headroom and isAllowedToWrap(M, x) returns true again.

A whitelisted address wraps M into that headroom and unwraps the minted USDat for the PYUSDx the drain supplied, re-establishing the M exposure the migration removes. USDat stays fully backed; the invariant is not enforced.

RECOMMENDATION

Reject M in USDat's _beforeWrap override instead of relying on cap == balance:

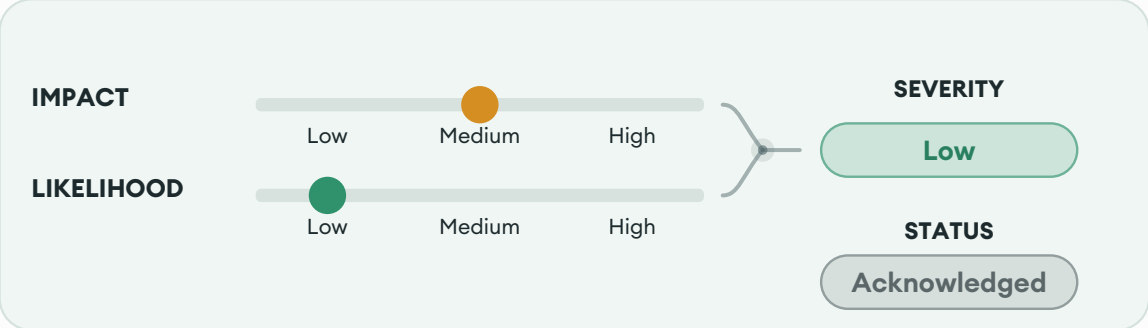
SOLIDITY

```
if (asset == M_TOKEN) revert MWrapDisabled();
```

DEVELOPER RESPONSE

> *"We are acknowledging this issue. Residual re-wrap headroom during draining is accepted; on completion the asset-cap manager sets M's cap to 0, which permanently disables M-wraps via isAllowedAsset. The whitelist can also be used to ban some addresses that would use M to wrap. Also, we prefer to not add any M related code/condition in beforeWrap, since the migration's goal is to remove M from the contract."*

L02: destinationToken and recipient are never validated as well-formed addresses before an irreversible burn



LOCATION

PYUSDx/143aa82b/src/portal/oft/PortalOFTWrapper.sol:L168

RELEVANT SNIPPET

>_ PortalOFTWrapper.sol — SOLIDITY

```
166: function setDestinationToken(uint32 destinationEid, bytes32 destinationToken) external
        onlyRole(OPERATOR_ROLE) {
167:     if (destinationEid == 0) revert ZeroDestinationEid();
168:     if (destinationToken == bytes32(0)) revert ZeroDestinationToken();
169:
170:     PortalOFTWrapperStorageStruct storage $ = _getPortalOFTWrapperStorageLocation();
```

DESCRIPTION

setDestinationToken only checks **destinationEid != 0** and **destinationToken != bytes32(0)**, and **_sendToken**'s **_revertIfZeroRecipient** only checks **recipient != bytes32(0)**. Neither validates that the upper 96 bits are zero, i.e. that the value actually decodes to a valid Ethereum address via **TypeConverter.toAddress()**.

Portal._sendToken burns the user's PYUSDx before dispatching the cross-chain message, then packs both the operator-configured **destinationToken** and the caller-supplied **recipient** verbatim into the payload.

On the destination chain, **decodeTokenTransfer** calls **.toAddress()** on both fields, which reverts deterministically on every delivery attempt and every retry, since the payload bytes never change once dispatched. So a malformed **destinationToken** (operator error) or a malformed **recipient** (user error, e.g. a fat-fingered or wrong-format address) both burn funds with no destination-side recovery path.

Note that PYUSDx currently only supports Ethereum and Arbitrum.

RECOMMENDATION

Validate address shape for both fields before the burn: **isValidAddress(destinationToken)** in **setDestinationToken**, and **isValidAddress(recipient)** in **_sendToken** (or its **_revertIfZeroRecipient** check).

SOLIDITY

```
if (!TypeConverter.isValidAddress(destinationToken)) revert InvalidDestinationToken(destinatio
  nToken);
if (!TypeConverter.isValidAddress(recipient)) revert InvalidRecipient(recipient);
```

DEVELOPER RESPONSE

"Acknowledged. We're not adding format validation on the send path deliberately. The payload keeps recipient and destination token as bytes32 because the Portal is designed to support non-EVM chains"

L03: setBridgeChainId desyncs remotePeer from a chain's reassigned EID, causing misrouted sends

LOCATION

PYUSDx/143aa82b/src/portal/bridgeAdapters/BridgeAdapter.sol:L86-L89

RELEVANT SNIPPET

>_ BridgeAdapter.sol — SOLIDITY

```
84:
85: // Clean up old forward mapping if this bridge chain was mapped to a different interna
      l chain
86: uint32 oldInternalChainId = $.bridgeToInternalChainId[bridgeChainId];
87: if (oldInternalChainId != 0 && oldInternalChainId != chainId) {
88:     delete $.internalToBridgeChainId[oldInternalChainId];
89:     emit BridgeChainIdRemoved(oldInternalChainId, bridgeChainId);
90: }
91:
```

DESCRIPTION

setBridgeChainId maintains the **chainId**<->**bridgeChainId** (EID) bijection but never touches **remotePeer**, so reassigning a chain's EID, or orphaning another chain as a side effect of the bijection cleanup, leaves **remotePeer[chainId]** holding a peer address for the old network while **internalToBridgeChainId[chainId]** now points at a different one.

sendMessage/**quote** read both mappings independently keyed only by **chainId**, so a subsequent send goes to the new EID but addressed to the stale peer, misrouting the message on the correct destination network, while the orphaned chain reverts with **UnsupportedChain**/**UnsupportedSender** until the operator restores its mapping.

RECOMMENDATION

Clear **remotePeer** for both the reassigned and orphaned **chainId**s inside **setBridgeChainId**, or replace it with an atomic setter that takes **(chainId, bridgeChainId, peer)** together.

FIX REVIEW NOTES

Fixed according to the recommendation.
The team added **_removePeer** calls to both existing cleanup branches in **setBridgeChainId**.

5. Enhancement Opportunities

E01: Empty replacement whitelist leaves replaceAsset open after migration

LOCATION

saturn-dollar/31e1d84a/script/UpgradeUSDatBase.sol:L53-L57

RELEVANT SNIPPET

>_ UpgradeUSDatBase.sol — SOLIDITY

```
51:     }
52:
53:     /// @dev Build the call the timelock makes on execute: ProxyAdmin.upgradeAndCall(p
    roxy, impl, migrate()).
54:     function _buildUpgradeAndCallData(address impl) internal view returns (address pro
    xyAdmin, bytes memory payload) {
55:         proxyAdmin = Upgrades.getAdminAddress(USDAT_PROXY);
56:         payload = abi.encodeCall(IProxyAdmin.upgradeAndCall, (USDAT_PROXY, impl, abi.e
    ncodeCall(USDat.migrate, ())));
57:     }
58: }
```

DESCRIPTION

USDat.migrate() registers the complete M balance as an allowed and replaceable asset, however, it does not add an authorized reserve operator to **replaceAssetWhitelist**. The upgrade payload constructed by **_buildUpgradeAndCallData** only calls:

SOLIDITY

```
IProxyAdmin.upgradeAndCall, (USDAT_PROXY, impl, abi.encodeCall(USDat.migrate, ()))
```

No subsequent whitelist configuration is included in the migration scripts.

Since **_isCallerAllowedToReplaceAsset** permits every caller while the whitelist is empty, **replaceAsset** remains unrestricted until a separate configuration transaction is executed:

SOLIDITY

```
return whitelist.length() == 0 || whitelist.contains(caller);
```

POTENTIAL BENEFIT

Ensures that M replacement is restricted from the first post-upgrade state.

RECOMMENDATION

Consider configuring the authorized replacement caller within **migrate()**, or update the timelock script to atomically execute both the upgrade and **setReplaceAssetWhitelistCaller**.

FIX REVIEW NOTES

Fixed by adding propose and execute scripts that route **setReplaceAssetWhitelistCaller(SOLVER, true)** through the asset-cap-manager timelock.

E02: migrate() registers the M reserve without emitting an asset-registration event

LOCATION

saturn-dollar/31e1d84a/src/USDat.sol:L72

RELEVANT SNIPPET

>_ USDat.sol — SOLIDITY

```
70:
71: /// @inheritdoc IUSDat
72: function migrate() external reinitializer(2) {
73:     // NOTE: The held M must cover the M-backed portion `totalSupply - totalAssets`. M
    earning is
74:     //       left on, so this balance already includes all yield accrued up to this bl
    ock.
```

DESCRIPTION

migrate() writes **$.assets[M_TOKEN]** and increments **$.totalAssets** directly instead of calling **setAssetCap**, so it emits no event for the registration. Every other function that mutates this state (**setAssetCap**, **_wrapAsset**, **_replaceAsset**) emits **AssetCapSet**, **AssetWrapped**, or **AssetReplaced**.

Consumers that track registered assets or **totalAssets** from logs will not observe M's registration or the **totalAssets** increase, leaving their view inconsistent with on-chain state until a later event or direct read corrects it.

POTENTIAL BENEFIT

Log-only consumers properly observe M's registration and the **totalAssets** change.

RECOMMENDATION

Emit an explicit registration event inside **migrate()**:

SOLIDITY

```
$.assets[M_TOKEN] = Asset({cap: mBalance, balance: UIntMath.safe240(mBalance), decimals: 6});
$.totalAssets += mBalance;
emit AssetCapSet(M_TOKEN, mBalance);
```

FIX REVIEW NOTES

Fixed according to the recommendation. The **migrate()** function now emits an event on asset cap update.

E03: _revokeRole clears the LayerZero delegate on any OPERATOR_ROLE revocation, not just the current delegate's

LOCATION

PYUSDx/143aa82b/src/portal/bridgeAdapters/layerZero/LayerZeroBridgeAdapter.sol:L91-L99

RELEVANT SNIPPET

>_ LayerZeroBridgeAdapter.sol — SOLIDITY

```
89:
90:  /// @dev Clears the LayerZero Endpoint delegate when the OPERATOR_ROLE is revoked from
        the current delegate.
91:  function _revokeRole(bytes32 role, address account) internal override returns (bool) {
92:      bool revoked = super._revokeRole(role, account);
93:
94:      if (revoked && role == OPERATOR_ROLE) {
95:          ILayerZeroEndpointV2(endpoint).setDelegate(address(0));
96:      }
97:
98:      return revoked;
99:  }
100:
101: /* ============ External View/Pure Functions ============ */
```

DESCRIPTION

_revokeRole calls **ILayerZeroEndpointV2(endpoint).setDelegate(address(0))** whenever any account loses **OPERATOR_ROLE**, without checking that **account** is the actual current delegate via **ILayerZeroEndpointV2(endpoint).delegates(address(this))**.

Since **OPERATOR_ROLE** can be held by multiple addresses at once (backup operator, key rotation), revoking a non-delegate operator still zeroes the real delegate's authority to configure DVNs/libraries or clear stuck endpoint messages, until a remaining **OPERATOR_ROLE** holder calls **setDelegate** again.

POTENTIAL BENEFIT

Ensure that removing one operator (for example, a backup or an old key being rotated out) does not accidentally disable the delegate that is still actively managing LayerZero.

RECOMMENDATION

Only clear the delegate when the account being revoked is the current delegate:

SOLIDITY

```
function _revokeRole(bytes32 role, address account) internal override returns (bool) {
    bool revoked = super._revokeRole(role, account);
    if (revoked && role == OPERATOR_ROLE && ILayerZeroEndpointV2(endpoint).delegates(address(t
  his)) == account) {
        ILayerZeroEndpointV2(endpoint).setDelegate(address(0));
    }
    return revoked;
}
```

DEVELOPER RESPONSE

"This behavior is intentional – it was introduced to resolve a finding from a previous audit. LZ delegate is an arbitrary address chosen by the operator, not necessarily the operator's own account. A compromised operator can set the delegate to any address they control before the role revocation."

E04: setFallbackRecipient does not validate that the fallback recipient is unfrozen on PYUSDx

LOCATION

PYUSDx/143aa82b/src/portal/Portal.sol:L569-L577

RELEVANT SNIPPET

>_ Portal.sol — SOLIDITY

```
567:
568: /// @dev Sets the address that receives PYUSDX or PYUSDX Extension on the destination
       chain when the intended recipient is frozen.
569: function _setFallbackRecipient(address fallbackRecipient_) internal {
570:     if (fallbackRecipient_ == address(0)) revert ZeroFallbackRecipient();
571:
572:     PortalStorageStruct storage $ = _getPortalStorageLocation();
573:     if ($.fallbackRecipient == fallbackRecipient_) return;
574:
575:     $.fallbackRecipient = fallbackRecipient_;
576:     emit FallbackRecipientSet(fallbackRecipient_);
577: }
578:
579: /// @dev Generates a unique across all chains message ID.
```

DESCRIPTION

setFallbackRecipient only rejects **address(0)**, never checking **IFreezable(PYUSDx).isFrozen(fallbackRecipient)**, even though PYUSDx freezing is controlled by an independent **FREEZE_MANAGER_ROLE**.

If the fallback recipient becomes frozen, **receiveMessage**'s redirect for frozen recipients hits **PYUSDx.mint**/**transfer**, which revert on a frozen recipient

POTENTIAL BENEFIT

Prevents a frozen fallback address from bricking delivery.

RECOMMENDATION

Reject a frozen address in **_setFallbackRecipient**.

DEVELOPER RESPONSE

"Acknowledged, but we're not adding the check, because it wouldn't prevent the following scenario: the check runs only when the fallback recipient is set, and the address can be frozen at any point afterwards by the freeze manager, producing exactly the same failure with the check in place. Overall, we consider the possibility of the fallback recipient being frozen unrealistic."

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in Web3 audits.

The firm was built by a mix of experienced professionals from traditional enterprise security and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members whose publications have earned over 1,100 academic citations, and with the trust of projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the Web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is designed to provide thorough security assessments of Ethereum smart contracts. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your smart contracts operate securely and as intended.

1. Smart Contract Context and Architecture Analysis

Our auditors begin by deeply examining your Ethereum smart contract's documentation and intended functionality. We meticulously map out contract interactions, transaction processing flows, and state management logic. Special attention is given to understanding how your smart contract interfaces with critical EVM standards, such as ERC-20, ERC-721, or ERC-4626 tokens, and governance/timelock modules. Last but not least we check external integrations with other DeFi protocols and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for the EVM execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from EVM-specific issues, including but not limited to:

- Unauthorized state variable modification
- Improper Access Control, role checks (**onlyOwner**, **require(hasRole)**) or **msg.sender** verification
- Misuse of low-level calls (**call**, **delegatecall**) or proxy/upgradeability patterns (e.g., storage collisions)
- Incorrect use of external calls, including reentrancy vectors
- Risks stemming from proxy initialization logic or denial-of-service related to gas limitations

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Solidity-based Ethereum smart contracts. This involves a comprehensive line-by-line inspection of source code, focusing on common EVM vulnerabilities including, but not limited to:

- Missing or insufficient Access Control and Role Checks
- Inadequate msg.sender and transaction origin checks
- Incorrect handling of external calls and invocation privileges, particularly potential reentrancy risks
- Arithmetic and integer overflow or underflow errors
- Unsafe use of ABI Encoding/Decoding or low-level assembly
- Improper ERC-20/Token Handling (e.g., approval logic, flash loan vectors, token locking)
- Logic flaws in financial operations or state transitions
- Edge-case handling in function input validation
- Potential denial-of-service vectors related to gas usage and transaction execution

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with EVM best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the EVM development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, express or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time of the engagement.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.